

Objectives

Perform exploratory Data Analysis and Feature Engineering using `Pandas` and `Matplotlib`

- Exploratory Data Analysis
- Preparing Data Feature Engineering

Import Libraries and Define Auxiliary Functions

We will import the following libraries the lab

```
[2]: # pandas is a software library written for the Python programming language for data manipulation and analysis.
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install seaborn
import pandas as pd
#NumPy is a Library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection
import numpy as np
# Matplotlib is a plotting library for python and pyplot gives us a MatLab like plotting framework. We will use this in our plotter function to plot da
import matplotlib.pyplot as plt
#Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statisti
import seaborn as sns
```

```
Requirement already satisfied: pandas in c:\users\amr\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\amr\anaconda3\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\amr\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\amr\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\amr\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\amr\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: numpy in c:\users\amr\anaconda3\lib\site-packages (2.3.5)
Requirement already satisfied: matplotlib in c:\users\amr\anaconda3\lib\site-packages (3.10.6)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (4.53.0)
Requirement already satisfied: kiwisolver in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (1.4.7)
Requirement already satisfied: packaging in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (24.1)
Requirement already satisfied: pyparsing in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (3.1.4)
Requirement already satisfied: pillow in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (10.4.0)
Requirement already satisfied: python-dateutil in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (2025.2)
Requirement already satisfied: six in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (1.17.0)
Requirement already satisfied: tzdata in c:\users\amr\anaconda3\lib\site-packages (from matplotlib) (2025.2)
```

Exploratory Data Analysis

First, let's read the SpaceX dataset into a Pandas dataframe and print its summary

```
[4]: import pandas as pd
URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
df = pd.read_csv(URL)
df.head(5)
```

```
[4]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Long
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.5
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.5
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.5
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.6
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.5

First, let's try to see how the `FlightNumber` (indicating the continuous launch attempts.) and `Payload` variables would affect the launch outcome.

We can plot out the `FlightNumber` vs. `PayloadMass` and overlay the outcome of the launch. We see that as the flight number increases, the first stage is more likely to land successfully. The payload mass also appears to be a factor; even with more massive payloads, the first stage often returns successfully.

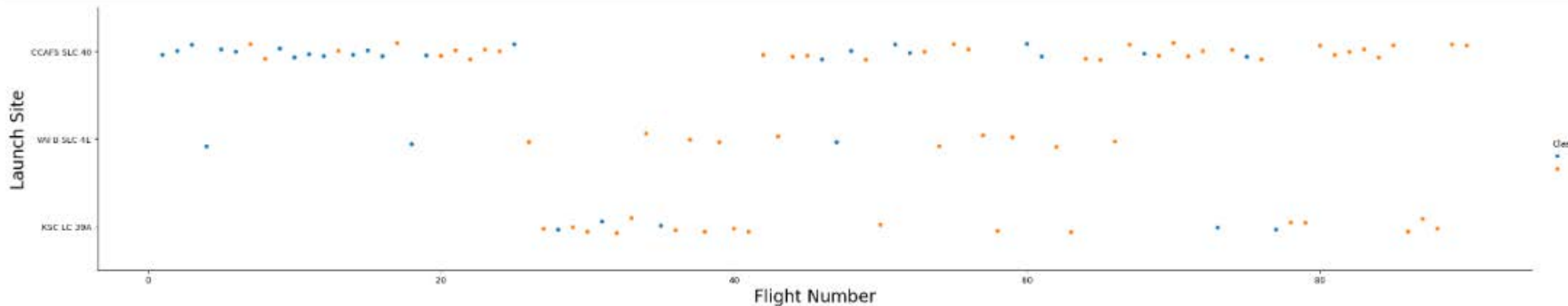
```
[ ]: sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect = 5)
plt.xlabel("Flight Number",fontsize=20)
plt.ylabel("Pay load Mass (kg)",fontsize=20)
plt.show()
```

Next, let's drill down to each site visualize its detailed launch records.

TASK 1: Visualize the relationship between Flight Number and Launch Site

Use the function `catplot` to plot `FlightNumber` vs `LaunchSite`, set the parameter `x` parameter to `FlightNumber`, set the `y` to `Launch Site` and set the parameter `hue` to `'class'`

```
[6]: # Plot a scatter point chart with x axis to be Flight Number and y axis to be the Launch site, and hue to be the class value
# TASK 1: Visualize the relationship between Flight Number and Launch Site
sns.catplot(
    x="FlightNumber",
    y="LaunchSite",
    hue="Class",
    data=df,
    aspect=5,
    kind="strip",
)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)
plt.show()
```

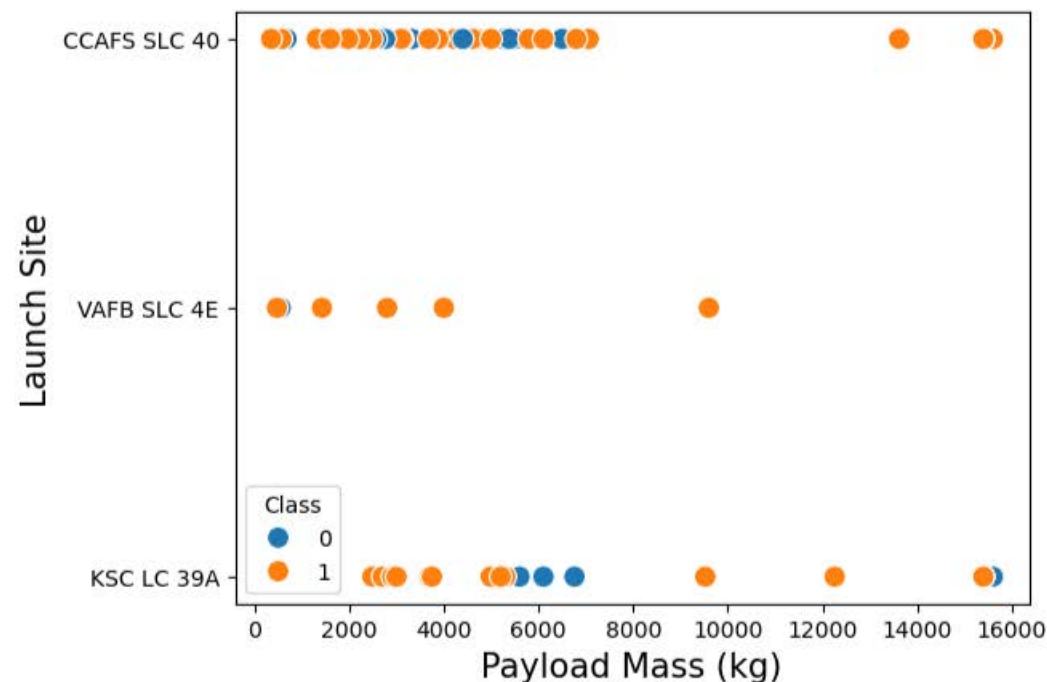


Now try to explain the patterns you found in the Flight Number vs. Launch Site scatter point plots.

TASK 2: Visualize the relationship between Payload Mass and Launch Site

We also want to observe if there is any relationship between launch sites and their payload mass.

```
[7]: # Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the Launch site, and hue to be the class value
sns.scatterplot(
    x="PayloadMass", y="LaunchSite", hue="Class", data=df, s=100
)
plt.xlabel("Payload Mass (kg)", fontsize=15)
plt.ylabel("Launch Site", fontsize=15)
plt.show()
```



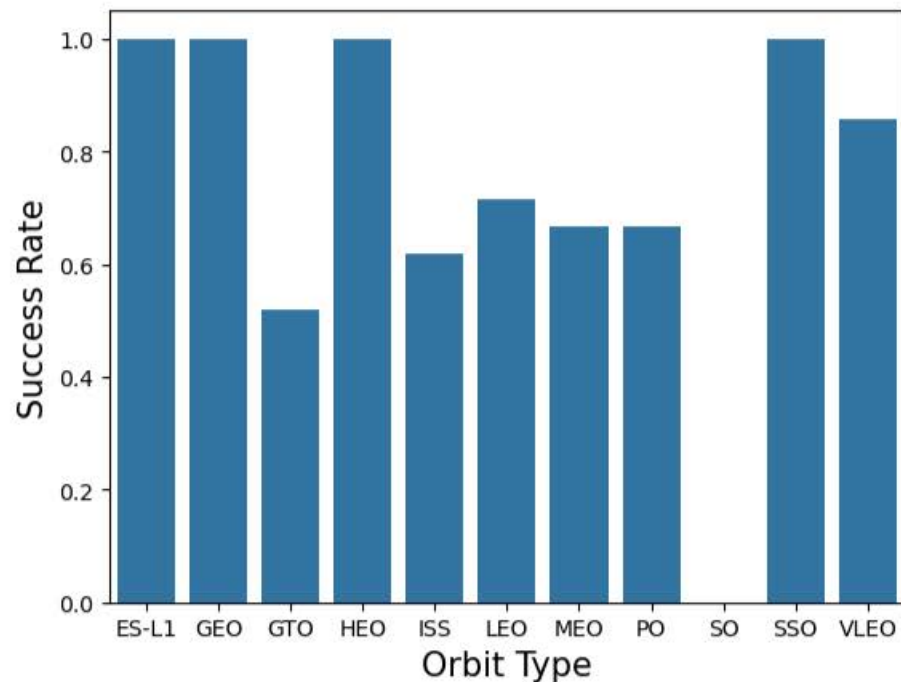
Now if you observe Payload Mass Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).

TASK 3: Visualize the relationship between success rate of each orbit type

Next, we want to visually check if there are any relationship between success rate and orbit type.

Let's create a `bar chart` for the success rate of each orbit

```
[8]: # HINT use groupby method on Orbit column and get the mean of Class column
# TASK 3: Visualize the relationship between success rate of each orbit type
orbit_success = df.groupby("Orbit")["Class"].mean().reset_index()
sns.barplot(x="Orbit", y="Class", data=orbit_success)
plt.xlabel("Orbit Type", fontsize=15)
plt.ylabel("Success Rate", fontsize=15)
plt.show()
```

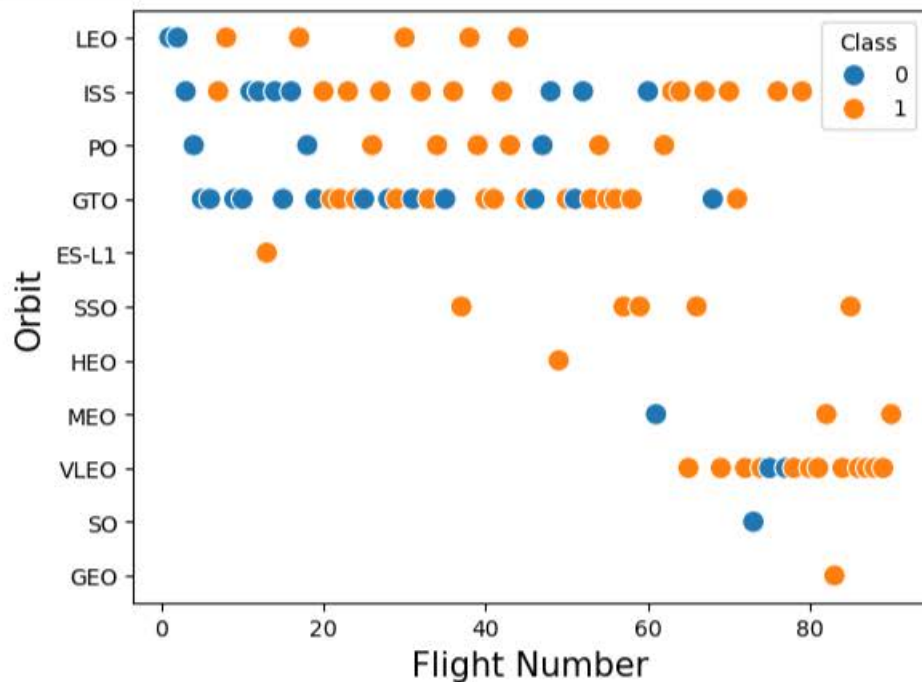


Analyze the plotted bar chart to identify which orbits have the highest success rates.

TASK 4: Visualize the relationship between FlightNumber and Orbit type

For each orbit, we want to see if there is any relationship between FlightNumber and Orbit type.

```
[9]: # Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
sns.scatterplot(x="FlightNumber", y="Orbit", hue="Class", data=df, s=100)
plt.xlabel("Flight Number", fontsize=15)
plt.ylabel("Orbit", fontsize=15)
plt.show()
```

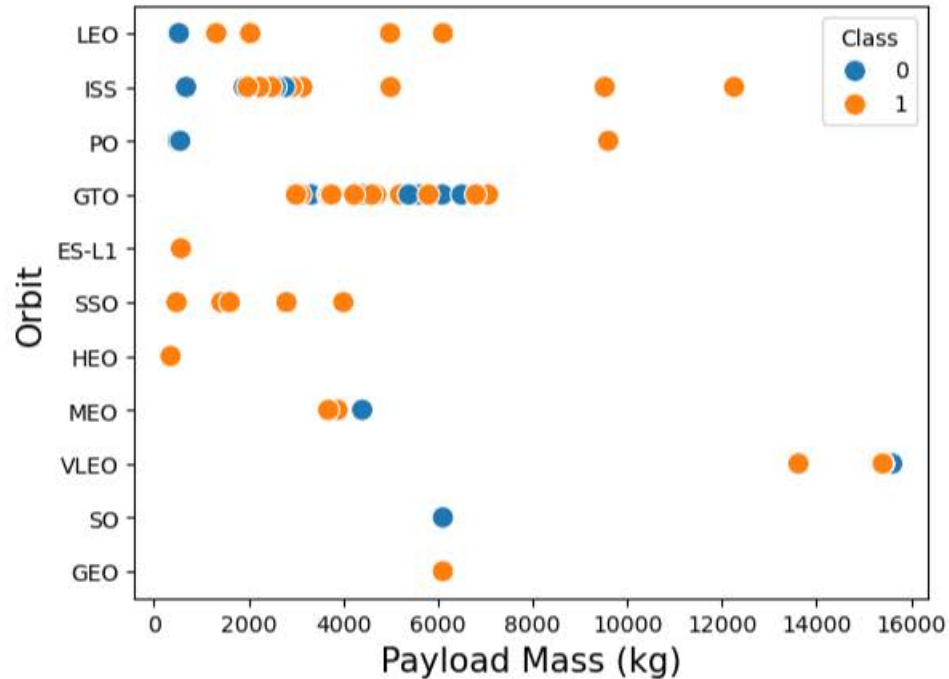


You can observe that in the LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.

TASK 5: Visualize the relationship between Payload Mass and Orbit type

Similarly, we can plot the Payload Mass vs. Orbit scatter point charts to reveal the relationship between Payload Mass and Orbit type

```
[10]: # Plot a scatter point chart with x axis to be Payload Mass and y axis to be the Orbit, and hue to be the class value
sns.scatterplot(x="PayloadMass", y="Orbit", hue="Class", data=df, s=100)
plt.xlabel("Payload Mass (kg)", fontsize=15)
plt.ylabel("Orbit", fontsize=15)
plt.show()
```



With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.

However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

TASK 6: Visualize the launch success yearly trend

You can plot a line chart with x axis to be `Year` and y axis to be average success rate, to get the average launch success trend.

The function will help you get the year from the date:

```
[13]: # A function to Extract years from the date
year=[]
def Extract_year():
    for i in df["Date"]:
        year.append(i.split("-")[0])
    return year
Extract_year()
df['Date'] = year
df.head()
```

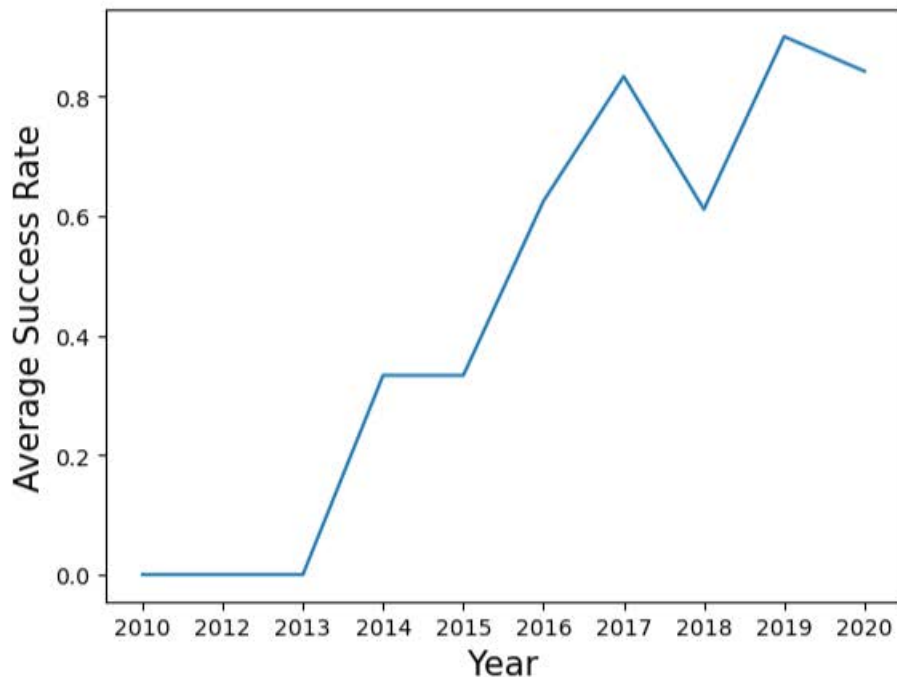
```
[13]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longi
0	1	2010	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.57
1	2	2012	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.57
2	3	2013	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.57
3	4	2013	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.61
4	5	2013	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.57

```
[15]: yearly_success = df.groupby("Date")["Class"].mean().reset_index()
sns.lineplot(x="Date", y="Class", data=yearly_success)
plt.xlabel("Year", fontsize=15)
plt.ylabel("Average Success Rate", fontsize=15)
plt.show()
```



```
[15]: yearly_success = df.groupby("Date")["Class"].mean().reset_index()  
sns.lineplot(x="Date", y="Class", data=yearly_success)  
plt.xlabel("Year", fontsize=15)  
plt.ylabel("Average Success Rate", fontsize=15)  
plt.show()
```



you can observe that the success rate since 2013 kept increasing till 2020

TASK 7: Create dummy variables to categorical columns

Use the function `get_dummies` and `features` dataframe to apply OneHotEncoder to the column `Orbits`, `LaunchSite`, `LandingPad`, and `Serial`. Assign the value to the variable `features_one_hot`, display the results using the method `head`. Your result dataframe must include all features including the encoded ones.

```
[16]: # HINT: Use get_dummies() function on the categorical columns
features = df[
    [
        "FlightNumber",
        "PayloadMass",
        "Orbit",
        "LaunchSite",
        "Flights",
        "GridFins",
        "Reused",
        "Legs",
        "LandingPad",
        "Block",
        "ReusedCount",
        "Serial",
    ]
]
features_one_hot = pd.get_dummies(
    features, columns=["Orbit", "LaunchSite", "LandingPad", "Serial"]
)
features_one_hot.head()
```

```
[16]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES- L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	Serial_B1051
0	1	6104.959412	1	False	False	False	1.0	0	False	False	...	False	False	False	False
1	2	525.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False
2	3	677.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False
3	4	500.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False
4	5	3170.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False

5 rows × 80 columns

TASK 8: Cast all numeric columns to float64

Now that our `features_one_hot` dataframe only contains numbers, cast the entire dataframe to variable type `float64`

```
[17]: # HINT: use astype function
features_one_hot = features_one_hot.astype("float64")
features_one_hot.dtypes
```

```
[17]: FlightNumber    float64
PayloadMass      float64
Flights          float64
GridFins         float64
Reused           float64
...
Serial_B1056     float64
Serial_B1058     float64
Serial_B1059     float64
Serial_B1060     float64
Serial_B1062     float64
Length: 80, dtype: object
```

We can now export it to a **CSV** for the next section, but to make the answers consistent, in the next lab we will provide data in a pre-selected date range.

```
features_one_hot.to_csv('dataset_part_3.csv', index=False)
```